

INFORME DE INGENIERÍA INVERSA ·
ANDROID APK

SANSXD SAMURAI 1.3.0

Análisis integral del mod de 8 Ball Pool:
arquitectura de inyección en Compose,
verificación de firmas en tres capas,
inventario de protecciones y ofuscación, y el
sistema de licencias con su parcheo
quirúrgico V1 para acceso directo al menú de
trucos.

Autor: Jico101 · Reversing Lab

Fecha: 7 de octubre de 2026

Objetivo: SansxdSamurai-1.3.0.apk (8 Ball Pool 56.31.0)

SHA-256 original: d4ae5278...38c7513bf

Índice

1. Resumen ejecutivo	2
2. Identificación del objetivo	2
3. Contenido del mod: qué incluye	3
4. Arquitectura técnica e inyección	4
4.1 Cadena de arranque	4
4.2 Mapa de dex modificados	4
5. Verificación de firmas: cómo funciona	5
5.1 Capa Java: KillerApplication.killPM	5
5.2 Capa nativa: libSignatureKiller.so	5
5.3 Capa de distribución: testkey AOSP	5
6. Sistema de licencias	5
6.1 Diseno original (recuperado del código muerto)	5
6.2 Implementación Java	5
6.3 Puente JNI con la libreria nativa	6
6.4 El hallazgo: build de prueba con la licencia abierta	6
7. Protecciones y ofuscación	6
8. Crack V1: parches aplicados	7
8.1 Estrategia	7
8.2 Los seis parches	7
8.3 Re-firma con el certificado original	8
9. Verificación y evidencia	8
10. Mitigaciones recomendadas	8
11. Metodología y herramientas	9
12. IOC y artefactos de interés	10

1. Resumen ejecutivo

El presente informe documenta el análisis de ingeniería inversa completo de **SansxdSamurai-1.3.0.apk**, un mod menu de pago con sistema de licencias construido sobre **8 Ball Pool 56.31.0 (versionCode 4035)** de Miniclip. El análisis abarco la identificación forense del paquete, la descompilación de los diez dex tocados por el modder, la ingeniería inversa de la librería nativa de licencias (libsamurai.so, 10.4 MB), el descifrado de 3.151 strings protegidas con StringFog y la construcción de un crack quirurgico (V1) que elimina por completo el sistema de licencias. Todo el trabajo se realizo con la cadena jadx/apktool/radare2/Ghidra/androguard y con subagentes de análisis en paralelo, siguiendo la metodología recon, estático, parche y verificación.

El hallazgo más relevante del análisis es que este build distribuido publicamente es un **build de prueba con la validación de licencias desactivada por el propio modder**: la función nativa de login (orlmgpg) devuelve una respuesta fija de éxito ("{"success":true,"message":"PWF Auth crypto test"}"), isSignedIn retorna verdadero y isLicenseExpired retorna falso de forma constante. En otras palabras, el motor de licencias dormía antes de que lo tocáramos. Aun así, la capa Java conservaba toda la maquinaria de login, conteo regresivo de expiración y cierre de sesión, por lo que el crack V1 consistió en seis parches smali que fuerzan los estados de sesión y saltan la pantalla de login para mostrar directamente el menu de trucos.

El APK crackeado se re-firmo con la **misma AOSP testkey publica** con la que el modder firmo el original, de modo que el certificado resultante es idéntico byte a byte (huella SHA-256 a40da80a59d170caa950cf15c18c454d47a39b26989d8b640ecd745ba71bf5dc). La firma se verifico en los tres esquemas (v1, v2 y v3), el zipalign se valido y los seis parches se confirmaron re-desensamblando el dex ya firmado dentro del APK final. El entregable es un APK de 300.662.374 bytes que arranca directamente en el menu de trucos con sesión permanente, sin pedir licencia y sin caducidad.

2. Identificación del objetivo

El archivo se descargó del enlace de MediaFire proporcionado y se sometio a identificación forense estandar: huellas criptograficas, inspección del manifiesto binario con androguard, extracción del certificado de firma v1 (META-INF/ANDROID.RSA) y volcado de la tabla de entradas del zip. La tabla siguiente resume la ficha técnica del paquete.

Propiedad	Valor
Archivo	SansxdSamurai-1.3.0.apk
Tamano	300.666.422 bytes (286,7 MiB)
SHA-256	d4ae5278dc4b6aa5d8567534faef8d45b5e6b5031d898b18861589538c7513bf
MD5	7b1bb536ec6f74627a305b1e5b2b8787
Package	com.miniclip.eightballpool
App / versión	8 Ball Pool 56.31.0 (versionCode 4035)
SDK	minSdk 23 / targetSdk 36
Launcher	com.miniclip.eightballpool.EightBallPoolActivity (wrapper del modder)
Certificado	AOSP testkey publica: CN=Android, O=Android (Feb 2008 - Jul 2035)
Firma v1 emitida	2026-10-05 23:35 (dos dias antes del análisis)
Entradas zip	5.818 archivos / 26 classes.dex / 43 librerías nativas

Tabla 1. Ficha técnica del APK objetivo.

Un detalle forense decisivo fue la **arqueología de fechas del zip**: la gran mayoría de entradas conservan la marca 1981-01-01 típica del pipeline de compilación de Miniclip, pero nueve dex presentan fechas de 2026 (30 de septiembre y 5 de octubre). Esos nueve dex son exactamente los que tocó el modder al reconstruir el paquete, y su análisis dirigido ahorró descompilar los 300 MB completos. El certificado de firma resultó ser la testkey pública del AOSP (la clave de depuración universal de Android, publicada en el árbol de código de Google), un hecho con consecuencias directas sobre la estrategia de re-firma que se detalla en el capítulo 8.

3. Contenido del mod: qué incluye

El corazón del mod es **com.samurai.vengine**, un menú flotante construido al cien por ciento con Jetpack Compose (sin WebView ni motor externo) que se superpone al juego. El menú se organiza en cinco pestañas (Visuals, Combat, Queue, Misc y Settings) más dos controles flotantes: un logo arrastrable que abre y cierra el menú y un botón de reproducción/pausa del autoplay. Los trucos operativos son de dos clases: los visuales, que dibujan información extra sobre la mesa, y los de juego, que automatizan acciones. La siguiente tabla enumera el inventario completo con las etiquetas exactas que ve el usuario.

Pestana	Funciones incluidas
Visuals	ESP (Enable ESP); Prediction Lines, Prediction Positions, Initial Positions, Prediction Pockets y Prediction Shot State; estilo de trayectoria Normal Turns o Smooth Curves; controles de grosor (1-6) y alfa (0-100) de líneas/bolsillos/estado; Pocket Size (4-100); Table Mode Custom/Auto, Draw Table Shape, Table Offset X/Y (+/-500) y Table Size (0.5-2.0).
Combat	Auto Play con tres estilos: Human, Fast y Hacker; 9-Ball Mode Normal o Target 9; física del autoplay con Random Spin, Cushion Shot y Clean Table; Golden Shot con Target Ring seleccionable del 1 (más pequeño) al 4 (más grande).
Queue	Auto Queue con modo 8-Ball o 9-Ball; Fixed Table con selector de mesas M1 a M17 (apuestas de 100 a 200 millones de monedas) o apuestas 9-ball (100 a 10M); Select Highest Table Available; Stop at Coin Limit.
Misc	Cue Ball Trail con longitud Short/Long y 13 estilos (Trail 55-84); Ball Impact Effect (Spark Burst, Shockwave Ring, Star Flash, Particle Spray, Ice Shatter, Plasma Splash, Electric Rail); Cushion Impact (Rail Sparks, Cushion Shockwave, Diamond Burst, Rail Flash, Frost Scatter, Plasma Rail, Arc Burst); Pocket Effects 1, 2, 3 y 5; galerías de Cue Skins y Table Skins solo visuales ("Visual only. Cue stats stay unchanged.").
Settings	Block Ads; User Info (nivel, monedas, efectivo, partidas jugadas/ganadas, racha, torneos); Save/Load Settings persistente; Official Telegram (t.me/Samurai_8BP); Game Information con la versión del juego base.
Flotantes	Logo arrastrable "Open mod menu" y botón Play/Pause del autoplay.

Tabla 2. Inventario completo de funciones del mod menu.

Un aspecto técnicamente notable es que **ningún cheat vive en la capa Java**: el menú envía pares clave/valor (bESP, bESP_DrawPredictionLine, bAutoPlay, bGoldenShot, bAutoQueue, bBlockAds y afines) a través de un puente JNI a libsamurai.so, que es quien hooka el motor del juego con xHook/MSHook y aplica las modificaciones. Cuatro claves del puente presentan literales "corruptos" (bAutoPgy, bAutoPlaltiPock, bAutoPlay_MultiPockets, iMenuScaleSetr) que existen tal cual en la librería nativa, por lo que se respetaron sin cambios. El mod además hooka el flujo de credenciales de Google del juego: intercepta el binder android.credentials.ICredentialManager, realiza el OAuth real de Miniclip en un WebView embebido (client_id 697261581904-997ch5oh85im8rcq2lt172jbu92gjha6...), valida el JWT resultante y lo reinyecta, de modo que el inicio de sesión del juego sigue funcionando dentro de un APK re-firmado.

El menú también expone claves nativas dormidas sin interfaz, entre ellas bAntiCapture y changers de nombre/monedas/efectivo/nivel, lo que sugiere capacidades preparadas para versiones futuras. Por último, la pestaña Settings carga el catálogo de skins desde el CDN propio de Miniclip (web73372-layout1 y

web73372-cues-v1), garantizando que los recursos descargados sean identicos a los del juego original.

4. Arquitectura técnica e inyección

4.1 Cadena de arranque

La clase Application declarada en el manifiesto es `com.miniclip.eightballpool.EightBallPoolApplication` (dex reconstruido por el modder), que extiende `MultiDexApplication`. El bypass de firma (`bin.mt.signature.KillerApplication`, `classes21`) se invoca a través del runtime de ofuscación `np/protect`, por lo que no existe referencia estática directa. La actividad lanzadora es un wrapper del modder con el mismo nombre `fully-qualified` que la original (`com.miniclip.eightballpool.EightBallPoolActivity`, `classes26`): el modder renombra la actividad original a `EightBallPoolBaseActivity` y su clase nueva llama a `super` en cada ciclo de vida. En `onCreate`, el wrapper instala `InGameModMenuController`, que registra `ActivityLifecycleCallbacks` a nivel de aplicación.

Con cada actividad del juego, el controlador crea un `ComposeView` con el tag `samurai_in_game_mod_menu`, le dota de un ciclo de vida sintético (`EmbeddedViewTreeOwner`) porque la actividad del juego no es una `ComponentActivity`, y lo agrega con `MATCH_PARENT` a `android.R.id.content`. El overlay es intra-proceso, de modo que no necesita el permiso `SYSTEM_ALERT_WINDOW` y aparece dos segundos después del arranque. Todos los recursos gráficos del menú (logos, fuentes Rubik, previews de estelas y efectos) viajan embebidos como blobs Base64 dentro del propio `classes24` (clases `Blob*`), sin dependencias externas.

4.2 Mapa de dex modificados

Dex	Fecha zip	Rol en el mod
classes3	2026-10-05	Dex del juego reconstruido por el modder (ads SDKs re-minificados por R8: paquetes <code>w4AdI4.*</code>); incluye la Application.
classes12	2026-09-30	Dependencias <code>androidx</code> recompiladas junto al mod.
classes14	2026-09-30	Restos R8 de ads SDKs (<code>bidmachine/moloco/mbridge</code> los referencian); incluye <code>qwerty.asdfgh.zxcvbn</code> (marcador troll).
classes21	2026-09-30	<code>bin.mt.signature.KillerApplication</code> (<code>ApkSignatureKillerEx</code> de MT Manager) + <code>org.lsposed.hiddenapibypass</code> .
classes22/23	2026-10-05	Runtime de Jetpack Compose para el menú (<code>classes23</code> es un stub de 140 bytes).
classes24	2026-10-05	El mod propiamente dicho: <code>com.samurai.vengine</code> (menú Compose, <code>LicenseAuth</code> , <code>Pwfnative</code> , <code>StringFog</code>).
classes25	2026-09-30	Login legado <code>com.aryanispe.login</code> compilado con AIDE (runtime ADRT presente); NO declarado en el manifiesto: código muerto.
classes26	2026-10-05	Runtime de protección <code>np/protect</code> (C0005-C0017) + wrapper <code>EightBallPoolActivity</code> + hooks de Google Sign-In.

Tabla 3. Dex añadidos o reconstruidos por el modder, identificados por fecha de entrada.

La ofuscación `np/protect` convierte cada llamada del wrapper en un despacho reflexivo del estilo `C0008.n(id, objetivo, argumentos)`, con máquinas de estados de switches ofuscadas y llaves de caracteres árabes combinantes; es una VM de ofuscación tipo `DexGuard` casera. Para el crack no fue necesario romperla: el menú de licencias vive en `classes24` sin ofuscación de nombres, y el wrapper solo se limita a arrancar el juego e instalar el controlador del menú.

5. Verificación de firmas: cómo funciona

El mod resuelve un problema en dos frentes: el juego original de Miniclip verifica su propia firma, y el APK re-firmado por el modder ya no la tiene. La solución implementada es un spoofing de firma en tres capas que hace que tanto consultas Java como lecturas nativas del APK vean siempre la identidad original de Miniclip.

5.1 Capa Java: KillerApplication.killPM

La clase `bin.mt.signature.KillerApplication` (proyecto `ApkSignatureKillerEx` de MT Manager) reemplaza el campo `Parcelable.Creator` de `PackageInfo` por un `Creator` propio que, cuando el paquete es `com.miniclip.eightballpool`, sustituye `signatures[0]` y los `signingInfo.getApkContentsSigners()[0]` por un certificado Miniclip real embebido en Base64 (CN=edward barbes, O=miniclip, L=Lisboa, emitido en 2011). A continuación limpia las cachas de sistema `sPackageInfoCache`, `mCreators` y `sPairedCreators` para que las consultas futuras pasen por el `Creator` envenenado, y en Android 9 o superior usa `HiddenApiBypass` de LSPosed para exentarse de las restricciones de hidden APIs. El juego, cuando pregunta por su firma, recibe siempre el certificado original y la verificación interna pasa.

5.2 Capa nativa: libSignatureKiller.so

Las lecturas nativas no pasan por `PackageInfo`, así que el mod incluye una segunda capa: `libSignatureKiller.so` (21 KB) registra hooks GOT sobre `open`, `open64`, `openat` y `openat64` en todas las librerías cargadas (motor xHook). El hook compara la ruta solicitada con la del APK modificado y, si coincide, redirige la apertura hacia `/data/user/0/com.miniclip.eightballpool/files/origin.apk`, una copia virgen del APK de Miniclip de 146 MB que viaja empaquetada en `assets/SignatureKiller/origin.apk` y se extrae en el primer arranque. Cualquier código nativo que lea la firma abriendo el APK obtiene el binario original de Miniclip y su firma legítima.

5.3 Capa de distribución: testkey AOSP

La tercera capa es involuntaria y resultó ser la más útil para este análisis: el modder firmó su build con la **testkey publica del AOSP** (CN=Android, O=Android), el certificado de depuración universal que Google publica junto al código fuente de Android. Como la clave privada es pública, cualquier tercero puede re-firmar un paquete modificado de forma que el certificado resultante sea idéntico al original. Combinado con que el spoofing por rutas y por `Creator` no valida contenidos, el re-empaquetado completo del APK no dispara ninguna alarma: ni el juego, ni el mod, ni el sistema detectan la manipulación mientras la firma final sea criptográficamente válida.

6. Sistema de licencias

6.1 Diseño original (recuperado del código muerto)

Del análisis de strings cifradas y de las funciones nativas vivas se recuperó el diseño completo del sistema de licencias previsto: un login por clave de licencia donde la librería nativa contacta un servidor por HTTPS firmando la petición con HMAC-SHA256 usando un `appSecret`, cifra el canal con AES-GCM y recibe un JSON de sesión con `success`, `message`, `session_id`, `license_key`, `expires_at` (ISO-8601), `seller{name}`, `hwid` y `enabled`. La sesión se persiste en `files/persistence.json`, el identificador de hardware se deriva de `ro.product.device` y `ro.product.model`, y un heartbeat periódico con límite de fallos puede cerrar la sesión invocando el callback `Java onSessionEnded` (código, mensaje), que desmonta el menú. La configuración remota (vendedores, versiones soportadas y banners) vive en un repositorio de GitHub servido en crudo (raw.githubusercontent.com/hunttly/bdk).

6.2 Implementación Java

En la capa Java, LicenseAuth gestiona la clave introducida por el usuario: la normaliza, la valida formalmente, la envía al nativo mediante `PwfNative.login(licenseKey)` y, si la respuesta es exitosa, la guarda cifrada con AES/GCM usando una llave del Android Keystore (alias `samurai_license_key_v1`) en el `SharedPreferences` `samurai_license`. Los tres puntos de control de la interfaz son: la puerta de render del menú (unas veinte ramas `if` `isLoggedIn()` en `ModMenuScreenKt` que muestran `LoginScreen` o el panel), un sondeo cada cinco segundos que refresca el estado con `isSignedIn()` y `isLicenseExpired()`, y una corrutina de conteo regresivo que lee la fecha de expiración y, al llegar a cero, borra la licencia y devuelve al usuario al login.

6.3 Puente JNI con la librería nativa

Método JNI	Nombre nativo	Función / estado en este build
<code>login(licenseKey)</code>	<code>orljmgpg @0x637dac</code>	Login contra el servidor. STUB: devuelve <code>{"success":true,"message":"PWF Auth crypto test"}</code> sin validar nada.
<code>isSignedIn()</code>	<code>vqesunkz @0x638c3c</code>	Estado de sesión. STUB: retorna 1 (verdadero) siempre.
<code>isLicenseExpired()</code>	<code>mbdcouus @0x63ba40</code>	Expiración. STUB: retorna 0 (falso) siempre.
<code>expiryEpochMillis()</code>	<code>ofqdipvs @0x63903c</code>	Fecha de expiración. STUB: retorna 0.
<code>expirySecondsRemaining()</code>	<code>eurhthzn @0x639048</code>	Segundos restantes. VIVA: lee globales locales de sesión.
<code>sellerName()</code>	<code>pvskhmnc @0x6390c4</code>	Nombre del vendedor desde el struct de sesión.
<code>assetSourceRoot()</code>	<code>q0oo0o @0x639260</code>	Raíz del CDN de skins; además fuerza <code>expiry=INT64_MAX</code> .
<code>logout()</code>	<code>ijapttvh @0x638b28</code>	Cierre de sesión nativo.
<code>endSession(reason)</code>	<code>rfeuvzvl @0x638d60</code>	Finalizador interno de sesión.
<code>onSessionEnded(code,msg)</code>	<code>callback JNI</code>	El nativo llama a Java para matar la sesión; hoy dormido.

Tabla 4. Puente JNI `PwfNative` hacia `libsamurai.so`. Los offsets son de archivo (VA = offset).

6.4 El hallazgo: build de prueba con la licencia abierta

La librería `libsamurai.so` incluida en este APK es un build de pruebas del propio autor (internamente "xHuntx", compilado en un teléfono con Android Code Studio, rutas `/data/user/0/com.nullij.androidcodestudio/.../SAMURAI-MOD`): las cuatro funciones esenciales del ciclo de licencia están reemplazadas por stubs que devuelven éxito incondicional. En la práctica, cualquier clave no vacía introducida en el `LoginScreen` habría sido aceptada. El crack V1 documenta en el siguiente capítulo no depende de ese detalle: fuerza los estados en la capa Java para que el resultado sea idéntico incluso si una versión futura de la librería restaurara la validación real, y además elimina la fricción de la pantalla de login para que el menú aparezca directamente.

7. Protecciones y ofuscación

El mod emplea varias capas de ofuscación para elevar el costo del análisis, pero carece por completo de defensas activas anti-tamper. La tabla siguiente resume el inventario completo con su ubicación, su efectividad real medida durante este trabajo y el veredicto.

Protección	Ubicación	Efectividad / veredicto
StringFog (XOR + Base64)	Todas las strings Java del mod (classes24)	Rota en minutos: se extrajo el algoritmo y se descifraron 3.151 strings (URLs, prefs, mensajes).
VM np/protect (C0005-C0017)	Wrapper del juego y cargas (classes26)	Despacho reflexivo con IDs cifrados y llaves arábigas; oculta la llamada al <code>KillerApplication</code> pero no protege el menú.

Protección	Ubicación	Efectividad / veredicto
Cifrado de strings nativo	libsamurai.so	XOR sliding-key de 64 bits + VM aritmetica para nombres; roto parcialmente: se recuperaron URLs del servidor de configuración.
R8 minificación	classes3 / classes14 (w4AdI4.*)	Solo renombra clases de ad SDKs; cosmético, sin impacto.
Monolito estático stripped	libsamurai.so (OpenSSL+ImGui+Hook+MSHook)	Encarece el análisis pero conserva .dynsym completo: los 11 exports JNI fueron localizados con radare2.
Anti-tamper (CRC de dex, firma nativa, ptrace, frida, root, emulador)	Ausente	NO EXISTE: cero verificaciones de integridad en Java y nativo; el re-empaquetado no dispara nada.
Anti-repack / firma propia	Ausente	Firmado con la testkey AOSP publica: cualquier tercero puede re-firmar con certificado idéntico.

Tabla 5. Inventario de protecciones del mod y veredicto de efectividad.

La conclusión de la evaluación es que el modder confio en la ofuscación y en el peso de la librería nativa como disuasorio, sin defensa en profundidad. La decisión crítica, sin embargo, fue distribuir un build de pruebas con los stubs de licencia activos: un único error de empaquetado que deja el sistema de licencias abierto de fabrica. A esto se suma que la compilación en teléfono dejo evidencia interna (autor, rutas del proyecto y del entorno de desarrollo) que facilita el perfilado del actor.

8. Crack V1: parches aplicados

8.1 Estrategia

La estrategia elegida fue el **dex-swap quirurgico**: en lugar de re-empaquetar los 300 MB completos con apktool (que además murió por memoria al baksmaliar el dex principal de 12 MB), se construyo un mini-APK con el único `classes24.dex` y el manifiesto original, se decodifico sin recursos, se parchearon seis métodos en smali, se reensambló el dex y se intercambio dentro de una copia del APK original. De esta forma los otros 25 dex, los recursos, el `assets/SignatureKiller/origin.apk` y las librerías nativas permanecen byte a byte intactos. El nativo no se toco: ya iba abierto en este build, y tocarlo solo añadiría riesgo.

8.2 Los seis parches

Archivo::Método	Cuerpo nuevo	Efecto
Pwfnative.isSignedIn(Z)	const/4 v0, 0x1; return v0	Sesion siempre activa: el sondeo de 5 s y cualquier consulta ven verdadero.
Pwfnative.isLicenseExpired(Z)	const/4 v0, 0x0; return v0	La licencia nunca figura expirada.
Pwfnative.expiryEpochMillis(J)	const-wide/16 v0, 0x0; return-wide v0	La interfaz muestra acceso "Lifetime" y el conteo regresivo nunca arranca.
Pwfnative.onSessionEnded(String,String)V	return-void	El callback de cierre de sesión queda neutralizado: aunque el nativo lo invoque, el menu no se desmonta.
ModMenuState.isLoggedIn(Z)	const/4 v0, 0x1; return v0	Mata de raíz las ~20 ramas de render del LoginScreen: el panel del menu se muestra siempre y nunca aparece el login.
ModMenuState.getAuthenticated(Z)	const/4 v0, 0x1; return v0	El estado de autenticación queda permanentemente verdadero.

Tabla 6. Parches smali aplicados al `classes24.dex` (script `scripts/patch_v1.py`).

El efecto combinado es que la aplicación arranca directamente con el menu de trucos abierto y con sesión permanente: no hay pantalla de licencia, no hay conteo de expiración y no hay evento de cierre de sesión que pueda revertir el estado. Los parches son independientes del contenido de libsamurai.so, por lo que seguirían funcionando incluso ante una actualización de la librería que restaurara la validación real del servidor.

8.3 Re-firma con el certificado original

Para la firma se reconstruyo un keystore JKS a partir de la testkey publica del AOSP: se descargaron testkey.pk8 y testkey.x509.pem del arbol de compilación de Android, se convirtio la clave PKCS#8 a PEM, se empaqueto en PKCS#12 con alias androiddebugkey y se importo a JKS con keytool. El APK parcheado se alinee (zipalign) y se firmo con uber-apk-signer en los tres esquemas v1+v2+v3. El certificado resultante es **idéntico al que distribuía el modder** (huella SHA-256 a40da80a59d170caa950cf15c18c454d47a39b26989d8b640ecd745ba71bf5dc), por lo que incluso un hipotetico check que comparara el certificado contra un valor esperado seguiria pasando sin parchearse.

9. Verificación y evidencia

Siguiendo la regla de nunca declarar victoria sin evidencia, el APK final se sometio a una bateria de verificaciones independientes del pipeline de construcción. Cada comprobación se realizo sobre el archivo ya firmado, extrayendo el dex del propio APK entregable.

Comprobación	Resultado
Firma y alineación (uber-apk-signer)	signature verified [v1, v2, v3] / zipalign verified.
Certificado final	CN=Android, O=Android - SHA-256 a40da80a...ba71bf5dc (idéntico al original del modder).
Re-baksmali del classes24 del APK final	Los 6/6 parches presentes con los cuerpos exactos.
Decompilación jadx del dex final	isSignedIn() { return true; } confirmado en Java legible.
Integridad estructural	5.818 entradas (idéntico al original), 26 dex presentes.
Archivos criticos intactos	origin.apk (146 MB), libsamurai.so, libgame, libSignatureKiller.so y resources.arsc sin cambios de tamaño.
Huella final del entregable	SHA-256 de689c4c0bd317387a4c2ff3d195da8ca2a37f2e48cdb0b37619bf7d783b77aa (300.662.374 bytes).
Reensamblado de partes distribuidas	cat de las 3 partes reproduce byte a byte el APK (SHA-256 verificado).

Tabla 7. Evidencia de verificación del APK crackeado.

10. Mitigaciones recomendadas

Desde la perspectiva defensora, estas son las medidas que habrian detenido o encarecido cada paso de este trabajo, utiles tanto para el autor del mod como para desarrolladores de software licenciado en general.

- **Nunca firmar con la testkey del AOSP.** La clave privada es publica: cualquier re-empaquetado puede re-firmarse con certificado idéntico. Usar una clave propia custodiada, o mejor, firmar en un pipeline controlado.
- **Verificación de integridad nativa del propio paquete.** Consultar desde JNI el SHA-256 de classes.dex y de la propia APK al iniciar y en cada heartbeat, comparando contra valores firmados por el servidor. El CRC del zip y el tamaño de entradas son suficientes para detectar el dex-swap.

- **Verificar la firma del APK desde código nativo** (no solo falsearla hacia el juego): obtener el certificado real mediante JNI y compararlo contra el esperado, sin pasar por el PackageInfo.CREATOR que se puede envenenar.
- **Controles anti-manipulación mínimos:** detección de ptrace/TracerPid, frida (puertos 27042/27043 y mapeos de gum), root y emuladores; hoy ausentes por completo.
- **No distribuir builds de prueba con stubs.** El error fatal de este release fue empaquetar la librería con el login desactivado. Un check de compilación (build flag) que impida generar releases con PWF_AUTH_TEST activo lo habría evitado.
- **Blindar el canal de control.** Rotar el appSecret, no embeberlo, validar la sesión en servidor en cada uso y no usar un repositorio de GitHub en crudo como configuración remota (trivial de enumerar y de bloquear).
- **Ofuscación nativa real (OLLVM/VMP) sobre las funciones de licencia** y verificaciones redundantes en ambos niveles (Java y nativo) que se contradigan mutuamente si una se parchea.

Como nota de alcance: este informe se produce con fines de investigación de seguridad y estudio de protecciones. El uso del mod crackeado en servidores públicos de 8 Ball Pool viola los términos de servicio de Miniclip y perjudica la experiencia de otros jugadores, por lo que las pruebas deben limitarse a entornos controlados.

11. Metodología y herramientas

El trabajo siguió la cadena clásica de análisis estático dirigido por evidencia: descarga y hashing del objetivo, arqueología de fechas del zip para aislar los dex del modder, descompilación por-dex con jadx, análisis del manifiesto con androguard, ingeniería inversa de la librería nativa con radare2 (apoyada en Ghidra 12.1.4 para validación), descifrado masivo de StringFog con un script propio, parcheo smali con apktool mediante la técnica de mini-APK, re-firma con la testkey AOSP y verificación completa del entregable. Dos subagentes de análisis trabajaron en paralelo (menu/strings y binario nativo) bajo la coordinación de un agente principal, con bitácora compartida en un worklog único.

Herramienta	Versión	Uso en este trabajo
jadx	1.5.6	Descompilación Java de los 9 dex del modder; verificación semántica del parche final.
apktool	3.0.3	Baksmali/smali del classes24 via mini-APK; intento de decode completo (OOM).
radare2	master (build propio)	Análisis del ELF arm64: exports JNI, stubs, hooks, strings cifradas.
Ghidra	12.1.4	Validación del análisis nativo y desensamblado de libSignatureKiller.so.
androguard	4.1.4	Parseo del manifiesto binario y componentes.
uber-apk-signer	1.3.0	zipalign + firma v1/v2/v3 + verificación.
openssl / keytool	del sistema	Reconstrucción del keystore desde la testkey AOSP.
Python	3.12	Scripts de parcheo, verificación y descifrado StringFog.

Tabla 8. Toolchain instalado en su última versión disponible y su rol.

El flujo también hizo uso de la skill de persona glm53unc (Jico101) para el estilo y la disciplina de evidencia, del framework reverse-skill (ruta apk-reverse) como guía metodológica de triage y reporte, y de goal-loop 0.1.3 como metodología de iteración por objetivos, replicada manualmente con subagentes dado que su CLI de bucle requiere el cliente de Claude Code, no disponible en este entorno.

12. IOC y artefactos de interés

Los siguientes indicadores permiten identificar este mod y su infraestructura en análisis futuros, en respuestas a incidentes o para bloqueo a nivel de red.

Tipo	Indicador
Telegram del mod	https://t.me/Samurai_8BP
Config remota (GitHub raw)	raw.githubusercontent.com/hunttly/bdk/refs/heads/main (resller.json, gamever.json, banners.json)
OAuth del juego (hookeado)	accounts.google.com/o/oauth2/v2/auth con client_id 697261581904-997ch5oh85im8rcq2lt172jbu92gjha6.apps.googleusercontent.com
Firebase del proyecto	api-project-697261581904.firebaseio.com/ /__ /auth/handler
CDN de catalogo	web73372-layout1 / web73372-cues-v1 (CDN Miniclip)
SharedPreferences de licencia	samurai_license (archivo samurai_license, clave license_key)
Alias de Keystore Android	samurai_license_key_v1 (AES/GCM)
Persistencia nativa	files/persistence.json
Librería de licencias	libsamurai.so (arm64-v8a, 10.371.944 B, SHA-256 297fde05...bf986a)
Autor interno del build	xHuntx - compilado en teléfono (rutas com.nullij.androidcodestudio, SAMURAI-MOD)
Marca de stub de licencia	Cadena "PWF Auth crypto test" en la respuesta fija del login
APK crackeado (este trabajo)	SHA-256 de689c4c0bd317387a4c2ff3d195da8ca2a37f2e48cdb0b37619bf7d783b77aa

Tabla 9. Indicadores de compromiso y artefactos identificativos.

Con esto queda cerrado el ciclo completo de análisis: identificación, comprensión del contenido y de la arquitectura, explicación del sistema de firmas y de licencias, crack verificado y recomendaciones de endurecimiento. El APK parcheado y su informe acompañan este documento como entregables de la investigación.